

SEMÁFOROS · asumo débiles salvo que diga	MONITORES · Mesa ($E = W < S$), sin spurious
señal $s=S(0)$ A: $x; s.rel()$ B: $s.acq(); y$	esquema: $while(!puedo) wait(c);$ cambio estado; $signal/All(c')$
rendez $a,b=S(0)$ A: $a.rel(); b.acq()$ B: $b.rel(); a.acq()$	$sem \rightarrow mon$ V: $x++$; $signal(cx)$ P: $while(x==0) wait(cx); x--$
mutex $m=S(1)$ · multiplex $s=S(K)$ · alternancia $pA=S(1)$ $pB=S(0)$	$thread\{ loop\{ m.empezar(); ACCIÓN LARGA; m.terminar(); \}$
LIGHTSWITCH (room, m, c)	BARRERA con fase
entrar: $m.acq; c++$; $if(c==1) room.acq; m.rel$	$esperar()\{ miF=fase; c++$;
salir: $m.acq; c--$; $if(c==0) room.rel; m.rel$	$if(c==n)\{ c=0; fase++$; $signalAll(cv) \}$
prior. lectores: lector = $LS(escibir)$ · escritor $escribir.acq/rel$	$else while(fase==miF) wait(cv) \}$
SIN INANICIÓN: mol = S(1, true)	TICKETS (orden de llegada)
lector: $mol.acq; mol.rel; LS.entrar \dots$ (de pasada)	$miT=prox++$; $while(miT!=turno \parallel !puedo) wait(c)$
escritor: $mol.acq; escribir.acq; mol.rel; E; escribir.rel$	$\dots$; $turno++$; $signalAll(c)$
PRIOR. ESCRITORES: leer, escribir, mutexL, mutexE, mutexP	LIBERAR DE A N (Atrapador)
lector: $mutexP.acq; leer.acq; LS_L.entrar(escibir)$;	$esperar: miT=prox++$; $signalAll(pLib)$; $while(miT>=lib) wait(at)$
$leer.rel; mutexP.rel; LEER; LS_L.salir(escibir)$	$liberar(N): while(prox-lib<N) wait(pLib)$; $lib+=N$; $signalAll(at)$
escritor: $LS_E.entrar(leer)$; $escribir.acq; E; escribir.rel$;	RENDEZVOUS (barbero)
$LS_E.salir(leer)$ (mutexP: 1 solo lector compite en leer)	$cli: esp++$; $signal(hayCli)$; $while(llam==0) wait(cL)$; $llam--$
PUENTE: mol.acq; LS[dir].entrar(res); mol.rel	$while(term==0) wait(cT)$; $term--$
cruzar (máx K: lugares.acq/rel alrededor); $LS[dir].salir(res)$	$empezar: while(esp==0) wait(hayCli)$; $esp--$; $llam++$; $signal(cL)$
BARRERA reutilizable	$terminar: term++$; $signal(cT)$
$m.acq; c++$; $if(c==n) t1.rel \times n; m.rel; t1.acq$	SALA / RONDAS
$m.acq; c--$; $if(c==0) t2.rel \times n; m.rel; t2.acq$	$asistir: while(enCurso \parallel lleno) wait(ent)$; $gente++$;
PROD-CONS: nF=S(N) nE=S(0) mP mC	$miCh=term$; $while(term==miCh) wait(fin)$
P: $nF.acq; mP\{ buf[in]=d; in++ \}$; $nE.rel$	$cerrar: enCurso=F$; $gente=0$; $term++$; $signalAll(fin)$; $signalAll(ent)$
C: $nE.acq; mC\{ d=buf[out]; out++ \}$; $nF.rel$	$secuenc.: while(t!=n) wait(c[n])$; $f()$; $t=(n+1)\%3$; $signal(c[t])$
k unidades: $mK.acq; k \times u.acq; mK.rel \dots k \times u.rel$ (sin mK)	MÁS PATRONES
PASO DE TESTIGO (FIFO sin fuertes)	$pizzas: while(g==0 \&\& ch<2) wait$; $if(g>0) g--$ else $ch-=2$
entrar: $m.acq; s=S(0)$; $p=fila.vacia$; $fila.add(s)$; $m.rel$;	no 2 seguidas: $while(ult==yo \&\& !fin) wait$; $ult=yo$; $\dots$; $signalAll$
$if(p) s.rel; s.acq$	bote: $bool$ subiendo + orilla; la autorización se gasta al partir
salir: $m.acq; fila.popFirst$; $sig=fila.peek$; $m.rel$; $if(sig) sig.rel$	recursos k: $while(cant<k) wait$; $tomar k$ · liberar: $signalAll$
BARBERO: m{ if lleno: me voy; else gente++ }	no perjudicar pedidos grandes: tickets, solo el primero toma
$cli: sol.rel; listo.acq; corte; term.acq; m\{gente--\}$; $ret.rel$	HOARE: signal cede ya $\Rightarrow$ while $\rightarrow$ if
$barb: sol.acq; listo.rel; corte; term.rel; ret.acq$	(si solo se señala con la cond cierta; $signalAll \Rightarrow$ sigue while)

EJ 1 · MUTEX · sin DEADLOCK/livelock · sin INANICIÓN (=GdE)SC termina; SNC puede no terminar; inanición = traza ∞ FAIR $x++ \equiv tmp=x; x=tmp+1$

I while(flag); flag=T; SC; flag=F → rompe MUTEX

II flag[i]=T; while(flag[o]); SC; flag[i]=F → DEADLOCK

III while(turno!=i); SC; turno=o → INANICIÓN

DEKKER

flag[i]=T;

while(flag[o]){ if(turno==o){ flag[i]=F; while(turno==o); flag[i]=T } }

SC; turno=o; flag[i]=F

PETERSON (2)

flag[i]=T; turno=o; while(flag[o] && turno==o); SC; flag[i]=F

mutex: el último en escribir turno espera · sin inanición

BAKERY (N)

eligiendo[i]=T; num[i]=1+max(num); eligiendo[i]=F

 $\forall j: \text{while}(\text{eligiendo}[j]); \text{while}(\text{num}[j]!=0 \ \&\& \ (\text{num}[j],j) < (\text{num}[i],i))$

SC; num[i]=0 (sin desempate por id → DEADLOCK)

ticket que se decrementa al salir → tickets repetidos → MUTEX

turno/proximo sin reset al salir el último → MUTEX (si reentran)

elegir siempre el menor id → INANICIÓN

traza: | T1 | T2 | estado (PC, vars locales) |

probar "vale": invariante o absurdo ("el último en entrar vio...")

(c) MODELO DE MEMORIA

todo lo anterior supone SC (interleaving)

x86-TSO: store buffer reordena Store→Load → Dekker $r1=r2=0$

HW: mfence (StoreLoad) o RMW (xchg, lock cmpxchg) · ARM: dmb, acq/rel

JMM: data race sin garantías · programas DRF $\Rightarrow$ SC

JIT cachea la variable → while(x!=id); nunca termina

Java: volatile (visibilidad + no reordena) · arrays: AtomicIntegerArray

funciones atómicas en Java → synchronized / Lock

lo que valía en (b): vale solo con volatile/fence

lo que fallaba: sigue fallando ($SC \subseteq TSO$) + motivos nuevos**EJ 4 · linealizable: PL dentro del intervalo, orden por PL válido**

CAS loop: while(T){ v=x.get(); if(x.CAS(v,v+1)) return }

immutable: while(T){ o=r.get(); if(r.CAS(o, new E(..))) return }

marcable: s=c.next.get(mk); c.next.CAS(s,s,F,T) (marcar)

sellada: o=top.get(st); top.CAS(o,n,st[0],st[0]+1)

FINA remove (hand-over-hand)

head.lock; pred=head; curr=pred.next; curr.lock;

while(curr.key<k){ pred.unlock; pred=curr; curr=curr.next; curr.lock }

if(curr.key==k) pred.next=curr.next $\leftarrow$ PL; unlock ambos

OPTIMISTA: recorrer sin lock; pred.lock; curr.lock; if(validate)...

validate: desde head, pred alcanzable && pred.next==curr

LAZY validate: !pred.m && !curr.m && pred.next==curr

remove: curr.marked=T $\leftarrow$ PL; pred.next=curr.next

contains: while(c.key<k) c=c.next; return c.key==k && !c.marked

LOCK-FREE find: curr marcado → pred.next.CAS(curr,succ) o retry

add: w=find; if(curr.key==k) F; pred.next.CAS(curr,n,F,F) $\leftarrow$ PLremove: curr.next.CAS(succ,succ,F,T) $\leftarrow$ PL; pred.next.CAS(curr,succ)**COLA 2 LOCKS**enq: enqL{ while(size==cap) wait; tail.next=e $\leftarrow$ PL; tail=e;

if(size.getAndInc()==0) wake ; if(wake) deqL{ notEmpty.All }

COLA MICHAEL-SCOTT

enq: last=tail; next=last.next; if(next==null){

if(last.next.CAS(null,n)) $\leftarrow$ PL { tail.CAS(last,n); return }

else tail.CAS(last,next) (helping)

deq: if(first==last){ if(next==null) throw $\leftarrow$ PL; tail.CAS(last,next) }else { v=next.val; if(head.CAS(first,next)) return v $\leftarrow$ PL }**PILA: push: n.next=top; top.CAS(old,n) $\leftarrow$ PL (+backoff)**pop: old=top; if(null) throw; top.CAS(old,old.next) $\leftarrow$ PLDOUBLE COLLECT: a=col; loop{ b=col; if(a==b) return Σ b; a=b }

swap: lock[min]; lock[max] · optimista: leer(v); computar; lock; if(ver==v)

wait-free toda · lock-free alguna · locks: deadlock/starv-free

lazy/LF contains wait-free · LF add/rem lock-free · opt/lazy: inanición

SEMÁFOROS — del enunciado al patrón	MONITORES — checklist Mesa
misimos juntos, excluyen a otro → LS por tipo + room	☐ todo wait dentro de un while (el despertado rechequea)
"a X no le molesta esperar" → prioridad al otro, sin molinete	☐ cada wait tiene quién lo despierte, DESPUÉS de cambiar el estado
"nadie espera para siempre" → LS + MOLINETE FUERTE compartido	☐ signal si los que esperan son intercambiables; si no signalAll
"si hay X esperando, los nuevos Y esperan" → X retiene mol (1 X)	☐ una sola condición → signalAll siempre
o LS de los X sobre `leer` (varios X)	☐ loops y acciones largas en el thread, no en el monitor
"esperar a N" / rondas → barrera de 2 molinetes	☐ ¿actualicé TODO el estado? (último, fin, contadores que bajan)
2+ recursos distintos → orden global / N-1 / romper simetría	☐ ¿resetear algo rompe el while de otro? → fase / número de ronda
k unidades del mismo pool → tomarlas bajo un mutex	☐ ¿un recién llegado se cuela y roba? si piden orden → tickets
"en orden de llegada" sin fuertes → paso de testigo	☐ ¿alguien retorna false y reintenta en vez de esperar? busy wait
"si está lleno, se va" → contador bajo mutex	☐ los que "esperan lo mismo", ¿esperan lo mismo? → separar
"combinación exacta de 2 cosas" → gestores + bools bajo mutex	☐ signal no tiene memoria: el hecho se guarda en una variable
hilo activo que controla (bote, barbero) → sem por rol que abre él	☐ al terminar el juego / cerrar: despertar a TODOS
CHECKLIST antes de entregar	DEL ENUNCIADO AL PATRÓN
☐ ¿alguien duerme con un lock que necesita quien lo despierta?	orden de llegada / no colarse → tickets (miT, turno)
☐ ¿mismo orden de acquire en todos? (condición antes que mutex)	liberar N juntos → liberadosHasta += N
☐ molinete: ¿retenido (frena) o de pasada (solo cruza)? ¿dónde?	rondas / "hasta que termine" → número de fase / charla
☐ LS antes o después del multiplex = quién cuenta para el otro	emparejar A con B → avisos por contador; varios → ticket de pareja
☐ ¿un permiso lo puede robar otro grupo? → uno por destino/rol	capacidad + tipos → while con condición compuesta
☐ ¿quién avisa y quién espera? el que hace la acción avisa	preferencia → ifs en orden de preferencia después del while
☐ ¿el int hace falta o ya lo cuenta un semáforo?	"no dos veces seguidas" → while(último==yo && !fin)
☐ fuertes donde piden sin inanición (mol, multiplex, mutex)	no perjudicar pedidos grandes → tickets: solo el primero toma
JUSTIFICAR (siempre)	JUSTIFICAR
cada sem: qué representa + valor inicial + invariante	mutex: lo da el monitor
mutex: #enSC + S.V = 1 ⇒ a lo sumo 1	sin deadlock: todo el que espera tiene quién lo despierte
sin deadlock: orden total · nadie duerme con lo que otro necesita	inanición: con tickets no hay · Mesa sin tickets: se cuelan
todo acquire tiene su release en todos los caminos	signal vs signalAll: decir por qué
inanición débil: traza fair donde siempre se cuela alguien	HOARE (inciso b)
fuerte: cota = los que ya estaban adelante en la cola	signal cede el monitor en el acto ⇒ la cond sigue cierta ⇒ if
escribirlo: "asumo débiles" / "este es fuerte porque..."	vale si solo se señala con la cond cierta y despierta uno
traza de sobrepaso: A espera en s; s.rel; B recién llega y	signalAll o señal "por las dudas" ⇒ sigue haciendo falta while
hace acq antes (barging); se repite ∞ → A nunca pasa	el que señala espera en la cola urgente (prioridad sobre entrada)

EJ 1 — procedimiento	EJ 4 — kit para regular
1 propiedades en orden: MUTEX → DEADLOCK → INANICIÓN (=GdE)	LINEALIZ.: cada op pasa de golpe (PL) dentro de su intervalo;
2 buscar: ¿marca antes o después de chequear? ¿turno? ¿x++? ¿for?	ordenando por PL se respeta el tiempo real y es válido
3 (a) no atómico: desagregar tmp=x; x=tmp+1 y entrelazar	SC no respeta tiempo real ni es composicional; lineal. sí
4 (b) atómico: cada llamada es 1 paso; rehacer el análisis	RECETA DEL PL
5 "vale": invariante o absurdo · "no vale": traza en tabla	1 función de abstracción (qué valor abstracto representa)
6 inanición: traza INFINITA y FAIR (un ciclo)	2 modifica: paso atómico donde cambia el valor abstracto
FALLAS TÍPICAS → qué rompen	3 lee o falla: la lectura que decide el resultado
chequeo antes de marcar → MUTEX (check-then-act)	4 separar éxito y falla
marcar antes de chequear, sin desempate → DEADLOCK/livelock	5 chequear: dentro del intervalo · atómico · estado coincide
turno estricto → INANICIÓN (el otro se queda en la SNC)	PLANTILLA: "op se lineariza en <línea>, porque ahí <cambio> ,
ticket decrementado / turno sin reset (si reentran) → MUTEX	y es atómico (<CAS / con lock>). op que falla en <lectura> .
elegir siempre el menor id → INANICIÓN de los ids altos	Ordenando por PL: ejecución válida de <spec> ⇒ linealizable"
dos llamadas atómicas separadas → otro se mete en el medio	NO linealizable: leer varios campos sin atomicidad
función con for no atómica → entrelazado dentro del for	PL puede ser un paso de otro thread (contains lazy que falla)
Peterson con otro=(id+1)%n, n>2 → MUTEX	PROGRESO
Bakery sin desempate por id → DEADLOCK	wait-free: toda op en pasos finitos (1 lectura, recorrer sin retry)
ref: Peterson flag[i]=T; turno=o; while(flag[o] && turno==o)	lock-free: alguna termina; mi CAS falla ⇒ el de otro tuvo éxito
(c) MEMORIA — respuesta modelo	deadlock-free: locks en orden total · starv-free: + locks fair
supuse SC. En la realidad: x86-TSO reordena Store→Load	optimista / lazy (validar y reintentar): NO starvation-free
(store buffer) y la JMM reordena/cachea si hay data races	RECETAS
lo que valía en (b) sigue valiendo solo si agrego:	granular fina: lock por parte, orden creciente; "todo": todos
lenguaje: volatile en compartidas (arrays: AtomicXArray)	lock-free: CAS loop · varios campos: immutable + AtomicReference
arquitectura: fence StoreLoad (mfence) o RMW atómico	double collect: iguales ⇒ snapshot (PL entre los dos)
lo que fallaba sigue fallando (SC ⊆ TSO) + motivos nuevos	si los valores bajan / reset → versiones por celda (ABA)
(visibilidad → while infinito, reordenamientos)	LISTAS PL éxito: add pred.next=n (LF: CAS) · remove lazy/LF: MARCAR
la arquitectura nunca arregla lo que ya falla bajo SC	fina (HOH): tomo el siguiente antes de soltar; lock del que borro
DEFINICIONES	cola 2 locks: PL enq tail.next=e · deq head=head.next
deadlock: todos esperan · livelock: giran sin progresar	despertar al otro bando tomando SU lock (no lost wakeup)
inanición (GdE): uno nunca entra aunque el sistema progrese	cola MS: PL enq CAS last.next · deq CAS head · helping de tail
fairness débil: instrucción continuamente habilitada se ejecuta	pila: CAS top · ABA sin GC → sello (AtomicStampedReference)
traza: paso T1 T2 vars (con PC y locales T1.tmp)	contador solo con inc de 1: get con 1 collect ya es linealizable

SUPUESTOS: sem débiles · Mesa · sin spurious · colas no fair	MONITORES (Mesa)
SEMÁFOROS enunciado → patrón	sem→mon V: x++; signal(cx) P: while(x==0) wait(cx); x--
misimos juntos excluyen a otro → LS por tipo + room	while(!puedo) wait; cambio estado; signal / signalAll
nadie espera ∞ → + MOLINETE FUERTE compartido	thread: loop + acción larga AFUERA; monitor = métodos cortos
X esperando frena a nuevos Y → X retiene mol / LS de X sobre leer	signal si son intercambiables · si no signalAll · 1 cond → All
rondas → barrera 2 molinetes · 2+ recursos → orden global	signal no tiene memoria → el hecho se guarda en una variable
k del mismo pool → bajo mutex · orden de llegada → testigo	no resetear lo que miran los while → FASE:
lleno se va → contador + mutex · combinación → gestores	miF=fase; c++; if c==n {c=0; fase++; All} else while(fase==miF) w
hilo activo (bote, barbero) → sem por rol que abre él (no LS)	TICKETS: miT=prox++; while(miT!=turno !puedo) w; ...; turno++; All
LS: m{ c++; if c==1 room.acq } ... m{ c--; if c==0 room.rel }	LIBERAR N: miT=prox++; while(miT>=lib) w · liberar: lib+=N; All
MOLINETE de pasada: acq; rel · retenido: acq ... rel después	liberador que espera: while(prox-lib<N) w(pLib) (esperar avisa)
sin inanición: esc: mol.acq; room.acq; mol.rel; E; room.rel	RENDEZVOUS en 3 avisos (barbero):
puente: mol.acq; LS[d].entrar; mol.rel (el 1ro trabado lo retiene)	cli: esp++; sig; while(llam==0) w; llam--; while(term==0) w; term--
multiplex después del LS · LS antes/después = quién cuenta	pel: while(esp==0) w; esp--; llam++; sig · terminar: term++; sig
PRIOR. ESC: lector mutexP{ leer{ LS_L.entrar(escribir) } }; LEER	varios peluqueros → emparejar (ticket)
esc: LS_E.entrar(leer); escribir.acq; E; rel; LS_E.salir(leer)	rondas de sala: número de charla; el que cierra pone gente=0
BARRERA: m{c++; if c==n t1.rel ×n}; t1.acq;	"no 2 seguidas": while(ult==yo && !fin) w · fin → All a todos
m{c--; if c==0 t2.rel ×n}; t2.acq	recurso k: while(cola<k) w; signalAll · justo: tickets
TESTIGO: m{ s=S(0); p=vacía; add(s) }; if p s.rel; s.acq	bote: bool subiendo + orilla; la autorización se gasta al partir
salir: m{ popFirst; sig=peek }; if sig sig.rel	CHECK
PERMISO ROBADO: sem compartido por grupos distintos	□ cada wait tiene quién lo despierte · actualicé todo el estado
→ uno por destino/rol (llegada[costa])	□ nadie hace return false para reintentar (busy wait)
el que hace la acción avisa "terminé"; el otro espera	□ ¿un recién llegado se cuela? si piden orden → tickets
CHECK	JUSTIFICAR: mutex automático · no deadlock: todo wait tiene
□ nadie duerme con un lock que necesita quien lo despierta	despertador · inanición: con tickets no; Mesa sin tickets sí
□ condición (notFull) ANTES que el mutex · k permisos bajo mutex	HOARE: signal cede ya ⇒ while → if (si solo se señala con la
□ fuertes: mol / multiplex / mutex si piden sin inanición	cond cierta y despierta uno) · signalAll / "por las dudas" ⇒ while
JUSTIFICAR: cada sem = qué es + invariante · mutex #SC+V=1	MÁS
deadlock: orden total / nadie duerme con lo que otro necesita	pizzas: while(g==0 && ch<2) w; if(g>0) g-- else ch-=2
inanición débil: traza fair con barging ∞ · fuerte: cota	secuenciador: while(t!=n) w(c[n]); f(); t=(n+1)%3; signal(c[t])
availablePermits nunca para decidir (check-then-act)	1 sola condición (inciso b) → todo signalAll
	Atrapador sin bloquear: if(prox-lib<N) return; lib+=N; All

EJ 1 · MUTEX → DEADLOCK → INANICIÓN (=GdE), en ese orden	EJ 4 — kit para regular
SC termina · SNC puede no terminar · inanición = traza ∞ FAIR	LINEALIZ.: cada op pasa de golpe (PL) dentro de su intervalo;
(a) no atómico: desagregar tmp=x; x=tmp+1 y entrelazar	ordenar por PL respeta tiempo real y es válido secuencialmente
(b) atómico: cada llamada = 1 paso · "vale": invariante/absurdo	RECETA PL
traza: T1 T2 estado (PC, vars locales)	1 función de abstracción · 2 modifica: donde cambia el valor
FALLAS TÍPICAS	3 lee/falla: la lectura que decide · 4 éxito ≠ falla
chequeo antes de marcar → MUTEX (check-then-act)	5 dentro del intervalo · atómico · el estado coincide
marcar antes de chequear, sin desempate → DEADLOCK/livelock	"op se lineariza en <línea> porque <cambio>; es atómico
turno estricto → INANICIÓN (el otro en la SNC)	(<CAS / lock>); ordenando por PL es válido ⇒ linealizable"
ticket decrementado / turno sin reset (si reentran) → MUTEX	leer varios campos sin atomicidad ⇒ NO linealizable
elegir siempre el menor id → INANICIÓN	PROGRESO
2 llamadas atómicas separadas / for no atómico → meterse en medio	wait-free: toda op finita · lock-free: alguna (CAS falla ⇒ otro ok)
Peterson (id+1)%n con n>2 → MUTEX · Bakery sin desempate → DEADLOCK	deadlock-free: locks en orden · starv-free: + locks fair
REFERENCIA	optimista / lazy (reintentan): NO starvation-free
Peterson: flag[i]=T; turno=o; while(flag[o] && turno==o); SC; flag[i]=F	RECETAS
Bakery: elig[i]=T; num[i]=1+max; elig[i]=F;	fin a N partes: lock por parte, orden creciente; "todo": todos
∀j: while(elig[j]); while(num[j]! = 0 && (num[j],j) < (num[i],i))	swap(i,j): lock[min]; lock[max] · PL con todos tomados
(c) MEMORIA — respuesta modelo	optimista: leer; computar sin lock; lock; validar versión; reintentar
supuse SC · x86-TSO: store buffer reordena Store→Load	CAS loop: while(T){ v=x.get(); if(x.CAS(v,v+1)) return }
JMM: con data races reordena y cachea (while(x!=id) infinito)	varios campos: objeto inmutable en AtomicReference + CAS
lo que valía: solo con volatile (arrays: AtomicIntegerArray)	double collect: 2 collects iguales ⇒ PL entre ambos
y fence StoreLoad (mfence) o RMW atómico (xchg, cmpxchg)	valores que bajan / reset → versiones por celda (ABA)
lo que fallaba: sigue fallando (SC ⊆ TSO) + motivos nuevos	PL DE ESTRUCTURAS
la arquitectura nunca arregla lo que falla bajo SC	listas: add pred.next=n (LF: CAS) · remove lazy/LF: MARCAR
funciones atómicas en Java = synchronized / Lock	falla: lock + validación ok · contains lazy: nodo sin marcar
DRF ⇒ SC (sin data races, la JMM garantiza SC)	cola 2 locks: enq tail.next=e · deq head=head.next
DEFINICIONES	cola MS: enq CAS last.next · deq CAS head · vacía: lee next
deadlock: todos esperan · livelock: giran sin progresar	pila: CAS top · ABA sin GC → AtomicStampedReference
inanición (GdE): uno nunca entra aunque el sistema progrese	lazy/LF contains wait-free · LF add/remove lock-free
fairness débil: toda instrucción continuamente habilitada corre	contador solo inc de 1: get con 1 collect ya linealizable